

## WEEK-10

### TASK-1: To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test 4 Signal Events

```
root@LAPTOP-RJFDBEST:~# nano signal_handlers.c
root@LAPTOP-RJFDBEST:~# ls -l signal_handlers.c
-rw-r--r-- 1 root root 2612 Sep 27 16:39 signal_handlers.c
root@LAPTOP-RJFDBEST:~# cat signal_handlers.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t signal_count = 0;

/* Signal handler */
void signal_handler(int sig, siginfo_t *info, void *context)
{
    signal_count++;

    printf("\n-----\n");
    printf("Signal Event Detected\n");
    printf("-----\n");

    printf("Signal Number : %d\n", sig);
    printf("Process ID : %d\n", getpid());
    printf("Signal Count : %d\n", signal_count);

    if (info != NULL)
    {
        printf("Sender PID : %d\n", info->si_pid);
    }

    switch (sig)
    {
        case SIGINT:
            printf("Signal Type : SIGINT (Interrupt)\n");
            break;
        case SIGTERM:
            printf("Signal Type : SIGTERM (Termination)\n");
            break;
        case SIGUSR1:
            printf("Signal Type : SIGUSR1 (User Signal 1)\n");
            break;
        case SIGUSR2:
            printf("Signal Type : SIGUSR2 (User Signal 2)\n");
            break;
        default:
            printf("Signal Type : Unknown\n");
    }

    printf("-----\n");
}

void register_handler(int sig)
{
    struct sigaction action;

    memset(&action, 0, sizeof(action));
    action.sa_sigaction = signal_handler;
    sigemptyset(&action.sa_mask);
    action.sa_flags = SA_SIGINFO;

    if (sigaction(sig, &action, NULL) == -1)
    {
        perror("sigaction");
        exit(1);
    }
}

int main()
{
    printf("Signal Handler and Context Management\n");
    printf("-----\n");

    printf("Process ID: %d\n", getpid());

    /* Register four signal handlers */
    register_handler(SIGINT);
    register_handler(SIGTERM);
    register_handler(SIGUSR1);
    register_handler(SIGUSR2);

    printf("\nSignal handlers registered successfully.\n");

    printf("\nConfigured Signals:\n");
    printf("SIGINT - %d - Ctrl+C Interrupt\n", SIGINT);
    printf("SIGTERM - %d - Termination Request\n", SIGTERM);
    printf("SIGUSR1 - %d - User Defined Signal 1\n", SIGUSR1);
    printf("SIGUSR2 - %d - User Defined Signal 2\n", SIGUSR2);

    printf("\nTesting Instructions:\n");
    printf("1. Press Ctrl+C for SIGINT\n");
    printf("2. Open another terminal and use kill -USR1 %d\n", getpid());
    printf("3. Open another terminal and use kill -USR2 %d\n", getpid());
    printf("4. Open another terminal and use kill -TERM %d\n", getpid());

    printf("\nProgram is waiting for signals...\n");

    while (1)
    {
        pause();
    }

    return 0;
}

root@LAPTOP-RJFDBEST:~# ./signal_handlers
Signal Handler and Context Management
-----
Process ID: 1518

Signal handlers registered successfully.

Configured Signals:
SIGINT - 2 - Ctrl+C Interrupt
SIGTERM - 15 - Termination Request
SIGUSR1 - 10 - User Defined Signal 1
SIGUSR2 - 12 - User Defined Signal 2

Testing Instructions:
1. Press Ctrl+C for SIGINT
2. Open another terminal and use kill -USR1 1518
3. Open another terminal and use kill -USR2 1518
4. Open another terminal and use kill -TERM 1518

Program is waiting for signals...

Signal Event Detected
-----
Signal Number : 2
Process ID : 1518
Signal Count : 1
Sender PID : 0
Signal Type : SIGINT (Interrupt)

Signal Event Detected
-----
Signal Number : 10
Process ID : 1518
Signal Count : 1
Sender PID : 0
Signal Type : SIGUSR1 (User Signal 1)

Signal Event Detected
-----
Signal Number : 12
Process ID : 1518
Signal Count : 1
Sender PID : 0
Signal Type : SIGUSR2 (User Signal 2)

Signal Event Detected
-----
Signal Number : 15
Process ID : 1518
Signal Count : 1
Sender PID : 1507
Signal Type : SIGTERM (Termination)

root@LAPTOP-RJFDBEST:~#
```

### TASK-2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt 4 Handling

```
root@LAPTOP-RJFDBEST:~# nano interrupt_handling.c
root@LAPTOP-RJFDBEST:~# ls -l interrupt_handling.c
-rw-r--r-- 1 root root 3981 Sep 27 16:41 interrupt_handling.c
root@LAPTOP-RJFDBEST:~# gcc interrupt_handling.c -o interrupt_handling
root@LAPTOP-RJFDBEST:~# ls -l interrupt_handling
-rwxr-xr-x 1 root root 16832 Sep 27 16:42 interrupt_handling
root@LAPTOP-RJFDBEST:~# ./interrupt_handling
SIGINT Forwarding and Foreground Job Control
=====
Shell PID : 1591
Shell PGID : 1591

SIGINT handler registered successfully.
Shell process is protected.

Foreground Job Information
=====
Job ID : 1
PID : 1592
PGID : 1592
Status : Running

Shell is protected from Ctrl+C.
Press Ctrl+C now to terminate the foreground job.

Foreground job started.
Child PID : 1592
Child PGID : 1592
Job is running...
Press Ctrl+C to interrupt the foreground job.
^C

Shell captured SIGINT (Ctrl+C).
Forwarding SIGINT to foreground job...
SIGINT forwarded successfully.
waitpid: Interrupted system call

Job Table
=====
Job ID PID Status Command
=====
1 1592 Running foreground_job
=====

Interrupt handling test completed.
root@LAPTOP-RJFDBEST:~#
```

### TASK-3: To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

[illegible]

## TASK-4: To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

```

root@LAPTOP-RJFD8EBT:~# nano process_group_control.c
root@LAPTOP-RJFD8EBT:~# gcc process_group_control.c -o process_group_control
root@LAPTOP-RJFD8EBT:~# ls -l process_group_control
-rwxr-xr-x 1 root root 16680 Sep 27 17:03 process_group_control
root@LAPTOP-RJFD8EBT:~# ./process_group_control
Process Group and Terminal Control Demonstration
=====
Shell PID      : 1680
Shell PGID    : 1680
Terminal PGID : 1680

Process Group Created
=====
Child PID : 1681

Child PGID : 1681
Child Process Created

=====
Transferring terminal control to child...
Terminal control transferred successfully.
Child PID : 1681
Current Terminal PGID: 1681
Child PGID : 1681

Child waiting for signals...
Sending SIGUSR1 to child process group...
SIGUSR1 sent to process group 1681.

Child received SIGUSR1.
Child PID : 1681
Child PGID : 1681

Child completed normally.

Sending SIGTERM to child process group...
SIGTERM sent successfully.

Child exited normally.

Restoring terminal control to shell...
Terminal control restored successfully.
Current Terminal PGID: 1680

Terminal ownership validation: SUCCESS

Process group control test completed successfully.
root@LAPTOP-RJFD8EBT:~#

```